Lecture 7

Word Representations

Traditional approach: One-Hot Encoding & Bag of Words

Numan Aslam

A model cannot multiply a word

A classifier multiplies inputs by weights and adds them up. The string “cat” has no product with a weight. So the first job of any NLP system is to turn each word into numbers — a vector.

```
cat   →   [ 1, 0, 0, 0, ... ]
```

Every method in this lecture is one answer to a single question: WHICH numbers? Today we cover the two simplest answers — one-hot encoding and bag of words.

Each word is one position, set to 1

```
Vocabulary:   { cat, dog, sat, mat }        V = 4 words
```

```
cat  =  [ 1, 0, 0, 0 ]
dog  =  [ 0, 1, 0, 0 ]
sat  =  [ 0, 0, 1, 0 ]
mat  =  [ 0, 0, 0, 1 ]
```

The vector length equals the vocabulary size. Each word gets a single 1 in its own slot, and 0 everywhere else. Simple — but watch what it costs on the next slide.

Every pair of words is equally unrelated

DOT PRODUCT MEASURES OVERLAP. FOR ONE-HOT WORDS IT IS ALWAYS ZERO:

$$\text{cat} \cdot \text{dog} = (1)(0) + (0)(1) + (0)(0) + (0)(0) = 0$$

$$\text{cat} \cdot \text{mat} = (1)(0) + (0)(0) + (0)(0) + (0)(1) = 0$$

So “cat” is EXACTLY as similar to “dog” as it is to “mat” — which is to say, not at all.

The representation carries no meaning. A model trained on “cat” learns nothing about “dog”, because the two share no overlap at all.

And it does not scale

FOR A REALISTIC ENGLISH VOCABULARY:

```
vocabulary size     =  50,000 words
each vector length  =  50,000 numbers
non-zero entries    =  1
zero entries        =  49,999
```

99.998% of every vector is zero.

Three problems, all inherited by the counting era: SPARSE (mostly zeros), HIGH-DIMENSIONAL (length = vocabulary), and SEMANTICALLY BLIND (no similarity).

A document = a vector of word counts

```
D1:  “the cat sat on the mat”
D2:  “the dog sat on the log”
```

Instead of one word, we now represent a whole document. We count how many times each vocabulary word appears. The order of the words is thrown away — hence “bag”.

Vocabulary (7 words):

```
[ the, cat, sat, on, mat, dog, log ]
```

We will count each of these in each document.

Counting, word by word

COUNT EACH VOCABULARY WORD IN EACH DOCUMENT

	the	cat	sat	on	mat	dog	log
D1	2	1	1	1	1	0	0
D2	2	0	1	1	0	1	1

```
D1  =  [ 2, 1, 1, 1, 1, 0, 0 ]
D2  =  [ 2, 0, 1, 1, 0, 1, 1 ]
```

“the” = 2 in both documents (it appears twice each). Note this — it is exactly what TF-IDF will punish later.

Order is thrown away

What it captures

Topic, reasonably well. A document about cats and mats produces a different vector from one about dogs and logs. Good enough for spam filters and topic sorting.

What it loses

Word order, entirely. These two sentences give the IDENTICAL vector:

“dog bites man”

“man bites dog”

The name is literal: the words are tipped into a bag and shaken.

Down-weight words that are everywhere

$$\text{weight}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

TF — term frequency

How often word t appears in document d (divided by the document length). A word repeated in a document is probably about that document.

IDF — inverse document frequency

log(N / df). Large when a word is rare across the collection, near zero when a word appears in every document.

Frequent HERE, rare ELSEWHERE → high weight.

A full calculation

N = 3 documents. D1 = “the cat sat on the mat” (6 words). log base 10.

```
the  : df=3   idf = log(3/3) = log(1) = 0
       tfidf(the, D1) = 0.333 × 0     = 0

cat  : df=2   idf = log(3/2) = 0.176
       tfidf(cat, D1) = 0.167 × 0.176 = 0.029
```

“the” scores 0 — it is in every document, so it cannot tell documents apart.

Put a little word order back

"the cat sat on the mat"

Unigrams (n=1)	`the · cat · sat · on · the · mat`
Bigrams (n=2)	`the cat · cat sat · sat on · on the · the mat`
Trigrams (n=3)	`the cat sat · cat sat on · sat on the`

Bigger n captures more order — but V=50,000 gives 2.5 BILLION possible bigrams. Almost all have count zero.

A bigram probability

P(next word | previous word) = count(pair) / count(previous)

In “the cat sat on the mat” :

count(the) = 2
count(the cat) = 1

P(cat | the) = 1 / 2 = 0.5

Given that we just saw “the”, there is a 50% chance the next word is “cat” — estimated from this text. Just a ratio of counts.

What all four methods share

1	**Sparse**	Almost every entry is zero.
2	**High-dimensional**	Vector length = vocabulary size.
3	**Order-limited**	Bag of words has none; n-grams only local.
4	**Semantically blind**	Nothing knows “doctor” ≈ “physician”.

The deepest problem is the last one. The fix: LEARN vectors from data — word2vec.

A word is known by its company

> *"You shall know a word by the company it keeps."* — *J. R. Firth, 1957*

"I poured a cup of ___ in the morning."

probably: tea or coffee

Because "tea" and "coffee" appear in similar contexts, they must mean similar things — so they should get similar vectors. This turns meaning (which we cannot see) into co-occurrence (which we can count).

Learn vectors from a fake task

Train a shallow network to predict words from their neighbours — then throw the network away and KEEP the hidden weights. Those weights are the word vectors.

CBOW

`context → centre`

Given the surrounding words, predict the missing middle word. Faster; better on frequent words.

Skip-gram

`centre → context`

Given the middle word, predict each surrounding word. Slower; better on rare words and small corpora.

The prediction is never used — the by-product (the weights) is what we wanted. This is self-supervision: the label comes free from the text.

Building training pairs from a window

“the quick brown fox jumps” centre = brown, window = 2

SKIP-GRAM : CENTRE → EACH CONTEXT WORD

```
(brown, the)
(brown, quick)
(brown, fox)
(brown, jumps)
```

CBOW : CONTEXT → CENTRE

```
input:
  { the, quick,
    fox, jumps }

target:  brown
```

Skip-gram makes 4 separate pairs; CBOW makes 1 example with all 4 context words at once. Slide the window right and “fox” becomes the next centre.

Dense, meaningful, and arithmetic

$$\text{vec(king)} - \text{vec(man)} + \text{vec(woman)} \approx \text{vec(queen)}$$

1	**Dense**	a few hundred numbers, not 50,000
2	**Meaningful**	similar words get similar vectors (cosine works)
3	**Relational**	“male → female” is a consistent direction
4	**Transferable**	train once on raw text, reuse everywhere

SUMMARY

Five ways to turn words into numbers

1	**One-hot**	sparse · no meaning · length = vocabulary
2	**Bag of words**	counts per document · loses word order
3	**TF-IDF**	weights down common words · strong baseline
4	**N-grams**	local word order · very sparse
5	**Word2Vec**	DENSE · learned · captures meaning

Counting counts words. Learning captures meaning. That is the whole story of this lecture.